

Московский авиационный институт

Институт 3. Кафедра 307

Лабораторная работа №1

Алгоритмы квадратичной сортировки и анализ эффективности

Выполнил: студент группы МЗО-114БВ-25

Орос Тимофей Иванович

2026 г.

Цель работы

Реализация и сравнение трёх вариантов пузырьковой сортировки, измерение временных и операционных затрат, анализ зависимости от размера и структуры входных данных.

Описание алгоритмов

Классический пузырьрёк

Алгоритм выполняет $(n-1)$ внешних проходов. На каждом проходе внутренний цикл сравнивает все соседние пары элементов от 0 до $n-2$ и переставляет их при нарушении порядка. Оптимизации отсутствуют — количество сравнений всегда равно $(n-1)^2$, независимо от исходного порядка данных.

Пузырёк с флагом

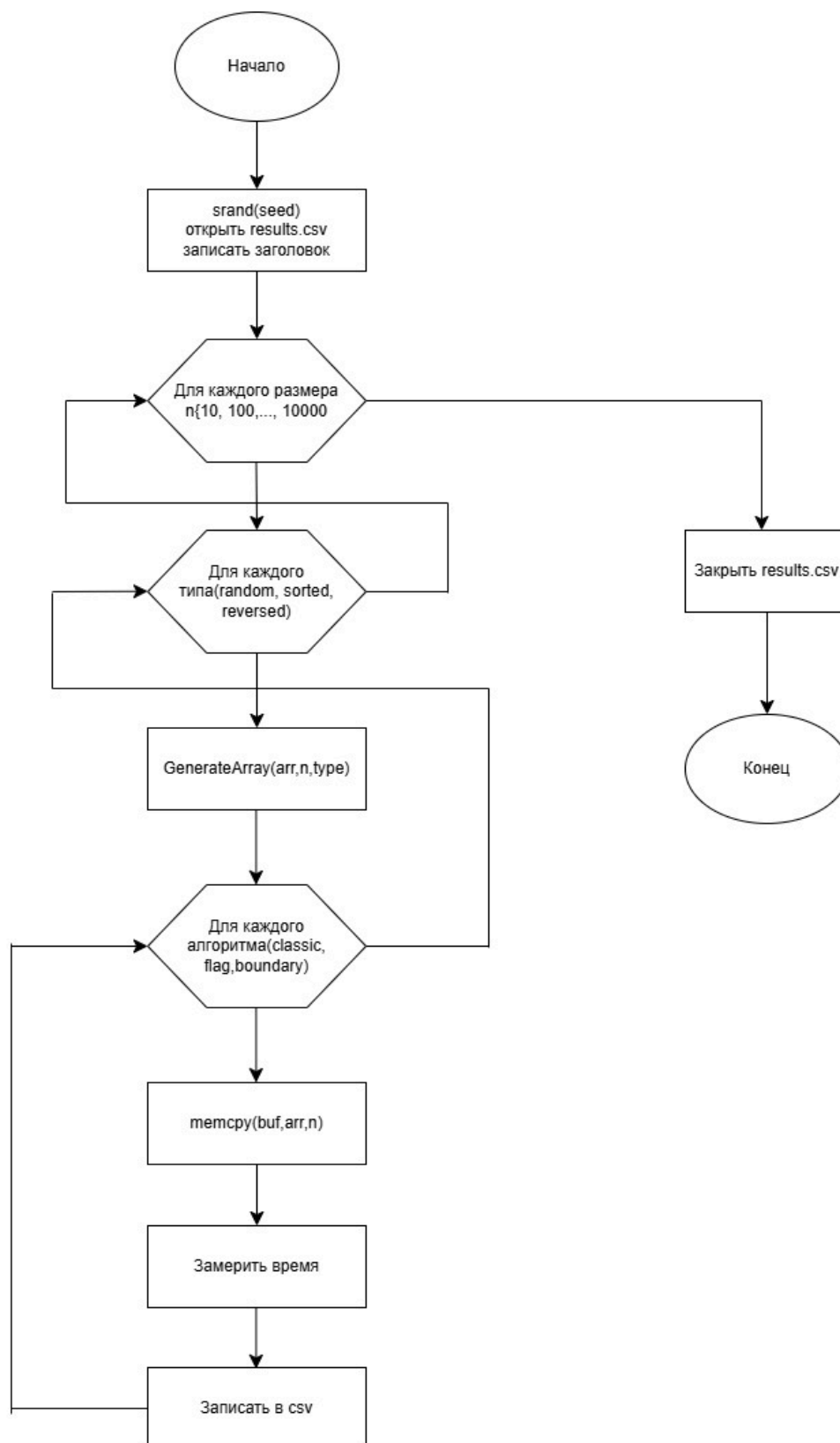
Перед каждым внешним проходом устанавливается флаг `swapped = false`. Если за весь внутренний проход не произошло ни одного обмена — массив уже отсортирован и алгоритм досрочно завершается. Это даёт выигрыш на частично или полностью отсортированных данных.

Пузырёк с уменьшением рабочей границы

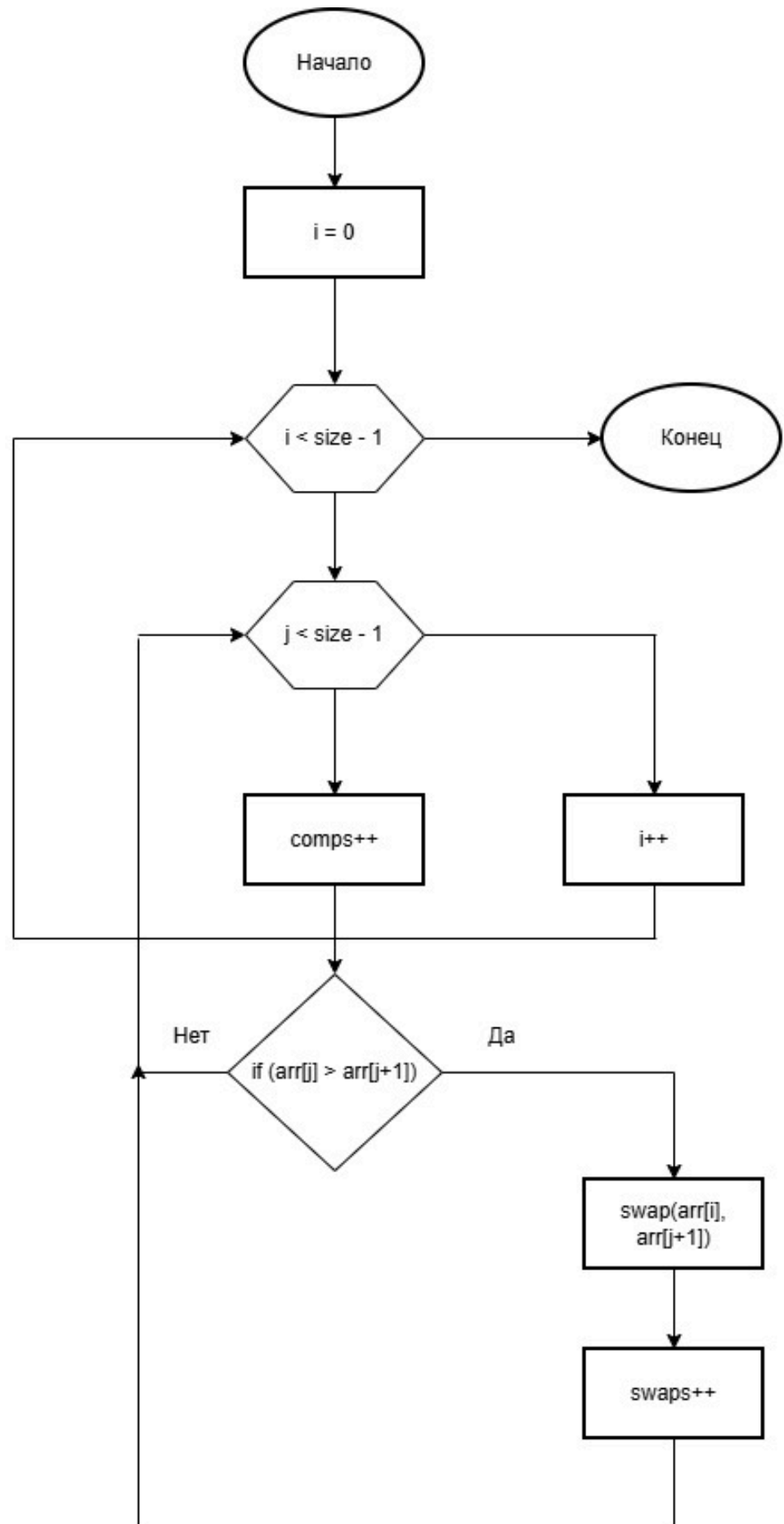
Запоминается индекс последнего обмена `lastSwap`. После каждого прохода граница внутреннего цикла сдвигается к этому индексу — элементы правее него уже стоят на своих местах и повторно не проверяются. Это уменьшает количество сравнений на каждой итерации.

Блок-схемы алгоритмов

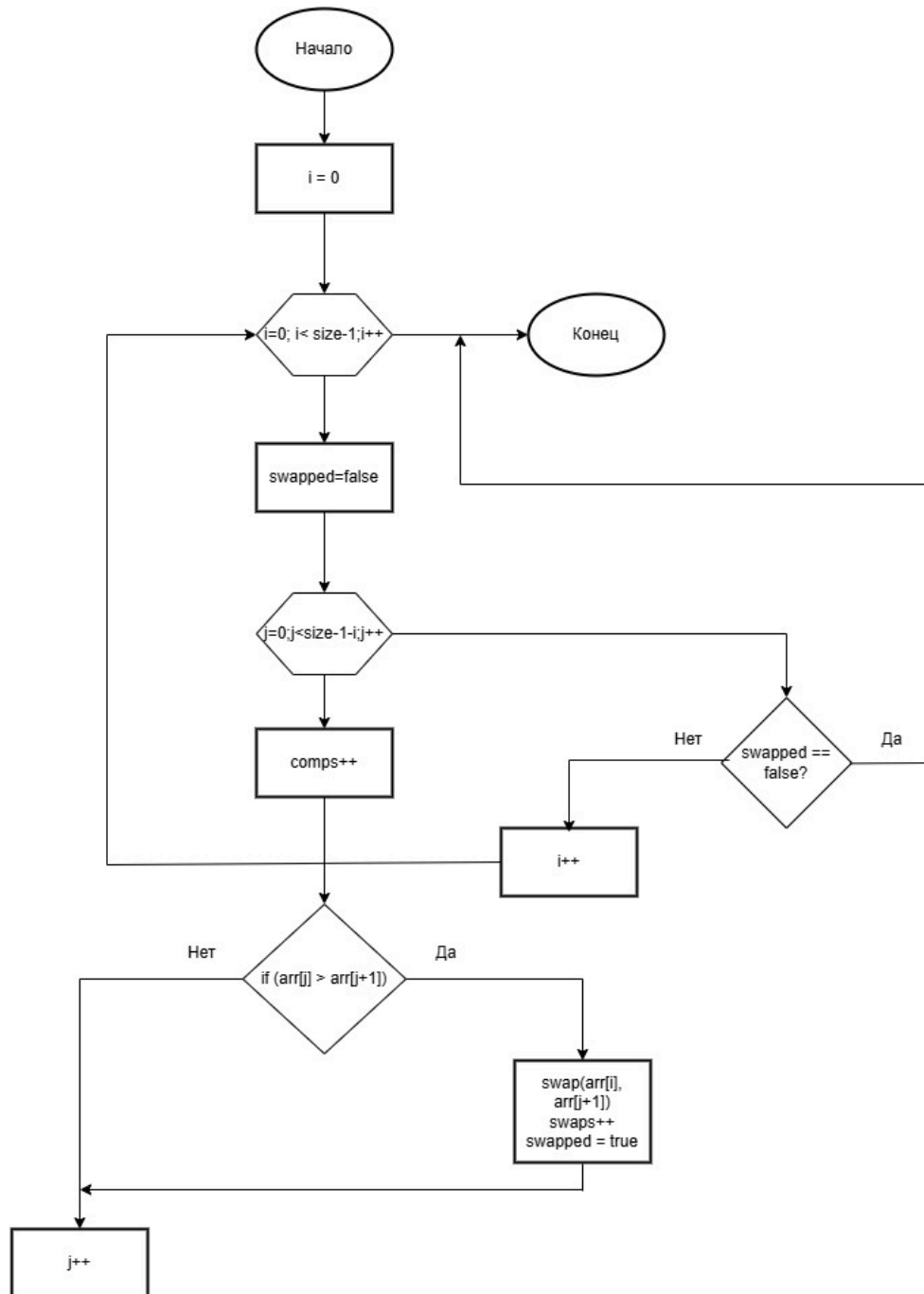
Блок-схема основной программы



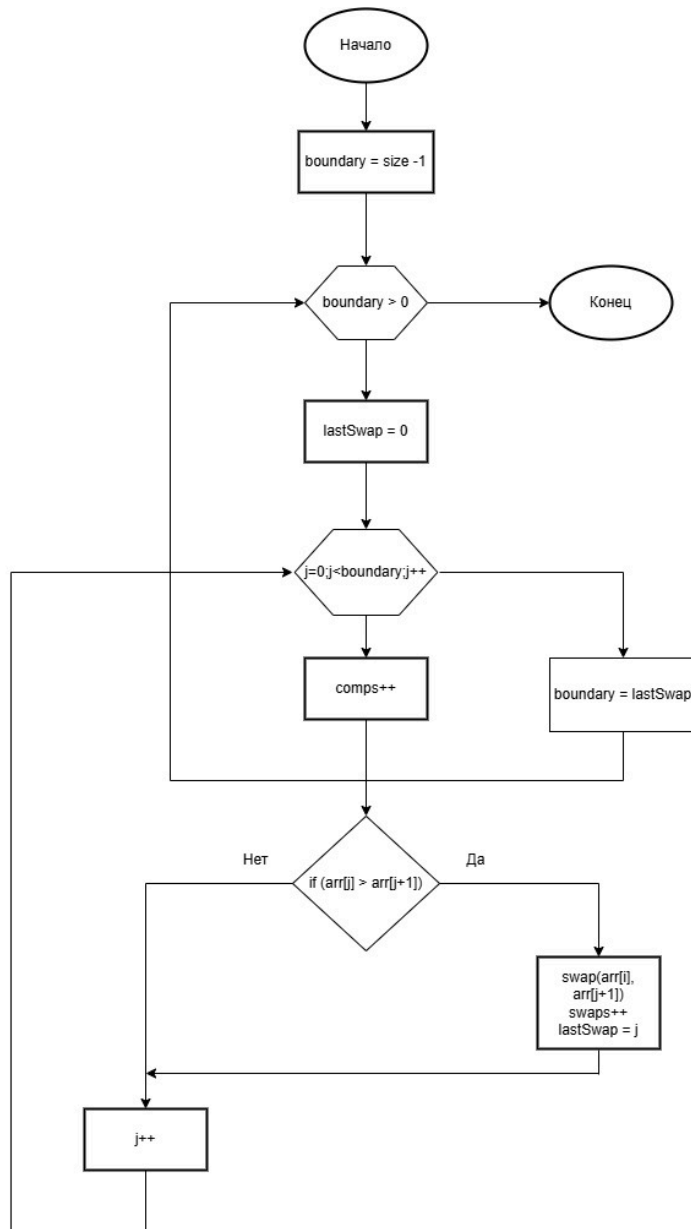
Блок-схема классического пузырька



Блок-схема пузырька с флагом



Блок-схема пузырька с динамической границей



Результаты экспериментов

Таблицы результатов

Случайные данные

n	Алгоритм	Сравнения	Обмены	Время (мкс)
10	boundary	40	24	0
100	boundary	4797	2416	0
500	boundary	122415	62524	0
1000	boundary	496463	243653	1997
2000	boundary	1997433	1004015	5616
5000	boundary	12463319	6135251	37407
10000	boundary	49979193	24977619	170464
10	classic	81	24	0
100	classic	9801	2416	0
500	classic	249001	62524	0
1000	classic	998001	243653	1997
2000	classic	3996001	1004015	7484
5000	classic	24990001	6135251	48541
10000	classic	99980001	24977619	212206
10	flag	42	24	0
100	flag	4922	2416	0
500	flag	124399	62524	996
1000	flag	499269	243653	998
2000	flag	1997622	1004015	6126
5000	flag	12481390	6135251	39144
10000	flag	49987250	24977619	181663

Отсортированные данные

n	Алгоритм	Сравнения	Обмены	Время (мкс)
10	boundary	9	0	0
100	boundary	99	0	0
500	boundary	499	0	0
1000	boundary	999	0	0
2000	boundary	1999	0	0
5000	boundary	4999	0	0
10000	boundary	9999	0	0
10	classic	81	0	0

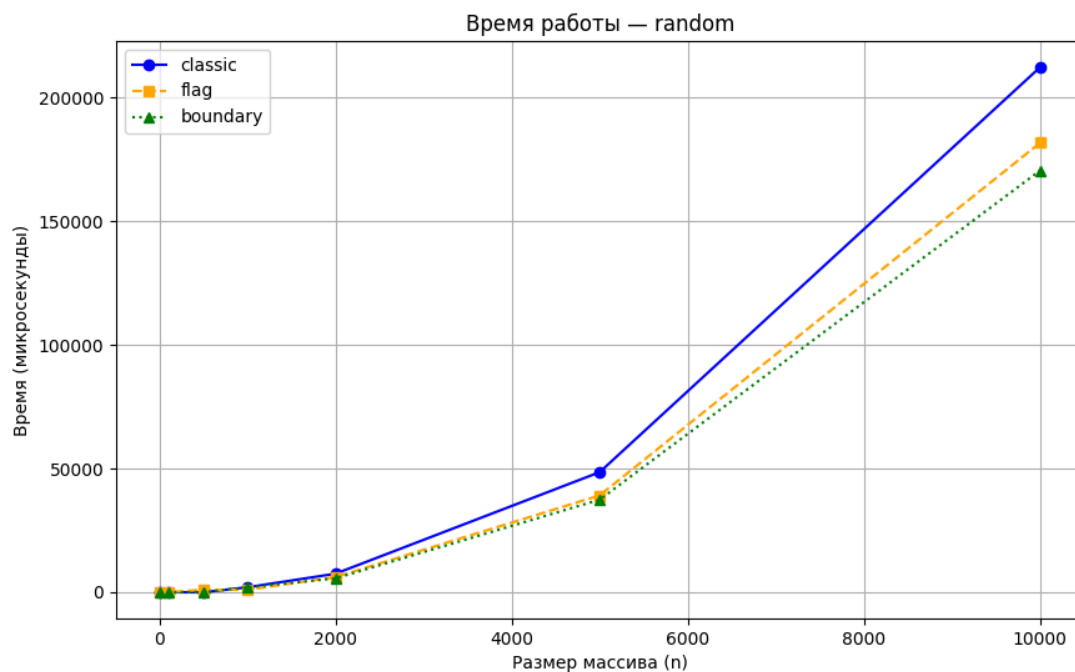
100	classic	9801	0	0
500	classic	249001	0	0
1000	classic	998001	0	998
2000	classic	3996001	0	4008
5000	classic	24990001	0	22760
10000	classic	99980001	0	94488
10	flag	9	0	0
100	flag	99	0	0
500	flag	499	0	0
1000	flag	999	0	0
2000	flag	1999	0	0
5000	flag	4999	0	996
10000	flag	9999	0	0

Обратный порядок

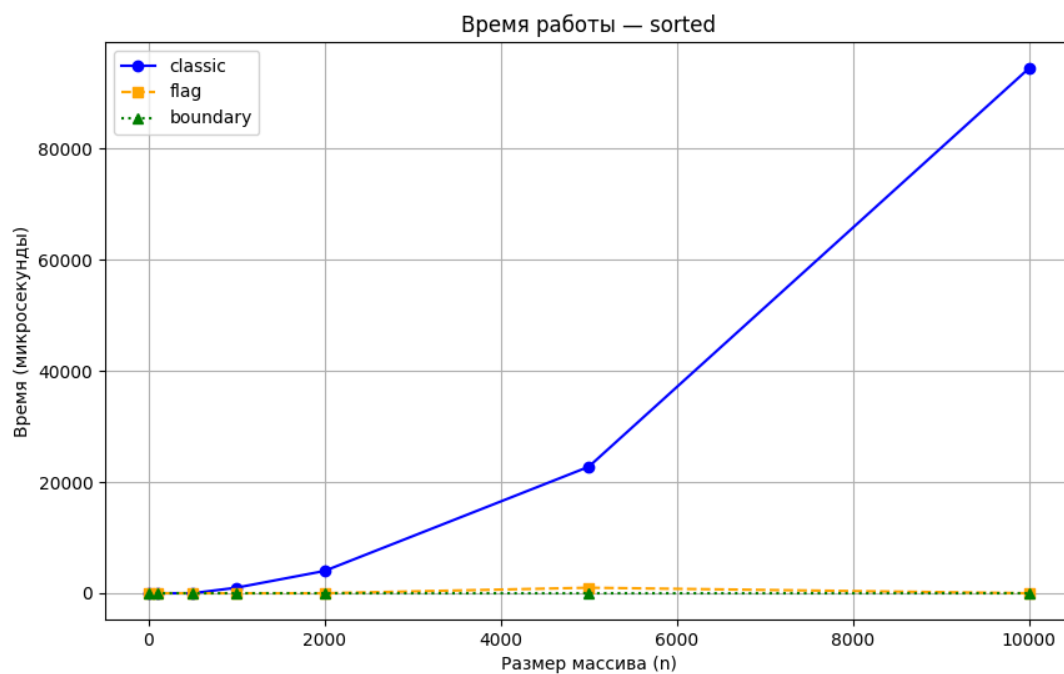
n	Алгоритм	Сравнения	Обмены	Время (мкс)
10	boundary	45	45	0
100	boundary	4950	4950	0
500	boundary	124750	124750	999
1000	boundary	499500	499500	2500
2000	boundary	1999000	1999000	6629
5000	boundary	12497500	12497500	44020
10000	boundary	49995000	49995000	173534
10	classic	81	45	0
100	classic	9801	4950	0
500	classic	249001	124750	1096
1000	classic	998001	499500	2115
2000	classic	3996001	1999000	8397
5000	classic	24990001	12497500	52695
10000	classic	99980001	49995000	211808
10	flag	45	45	0
100	flag	4950	4950	0
500	flag	124750	124750	0
1000	flag	499500	499500	1014
2000	flag	1999000	1999000	6969
5000	flag	12497500	12497500	43855
10000	flag	49995000	49995000	175376

Графики времени работы

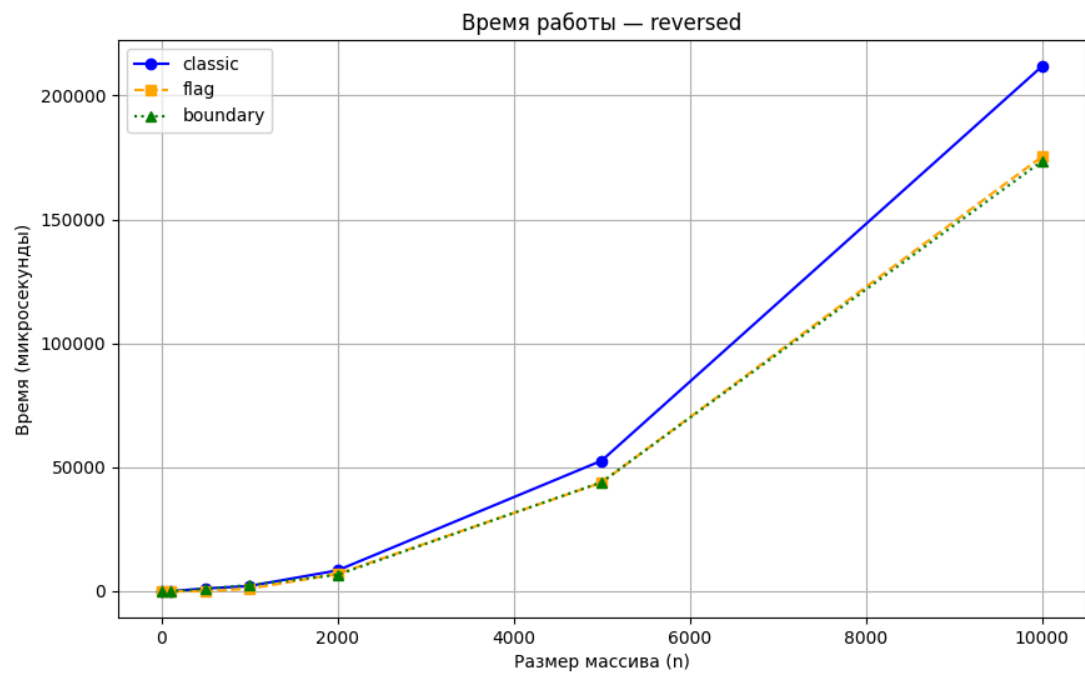
Случайные данные



Отсортированные данные

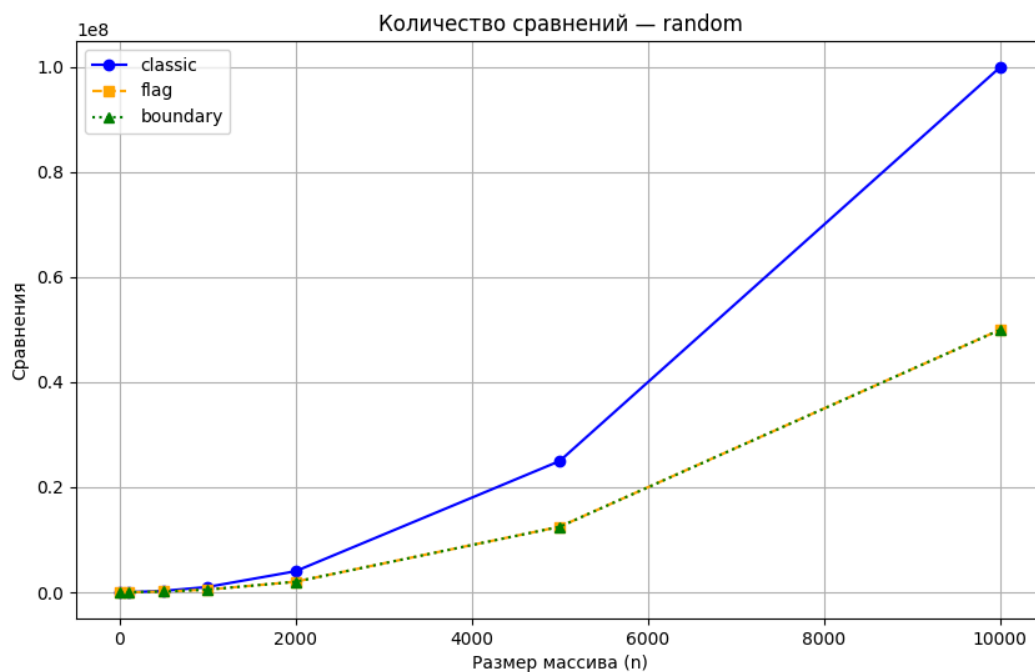


Обратный порядок

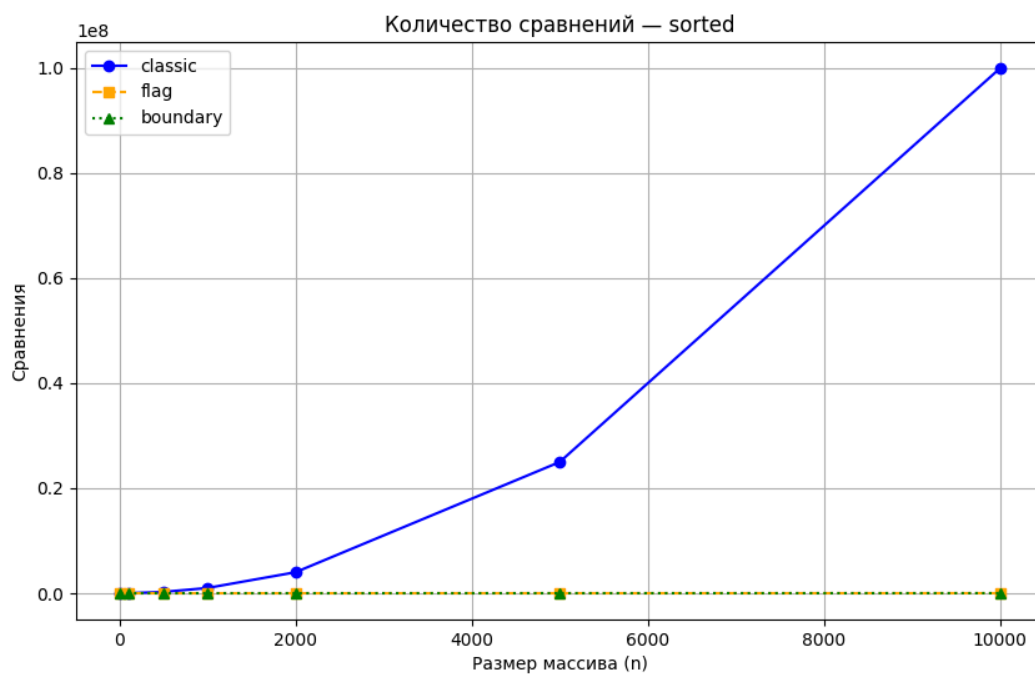


Графики количества сравнений

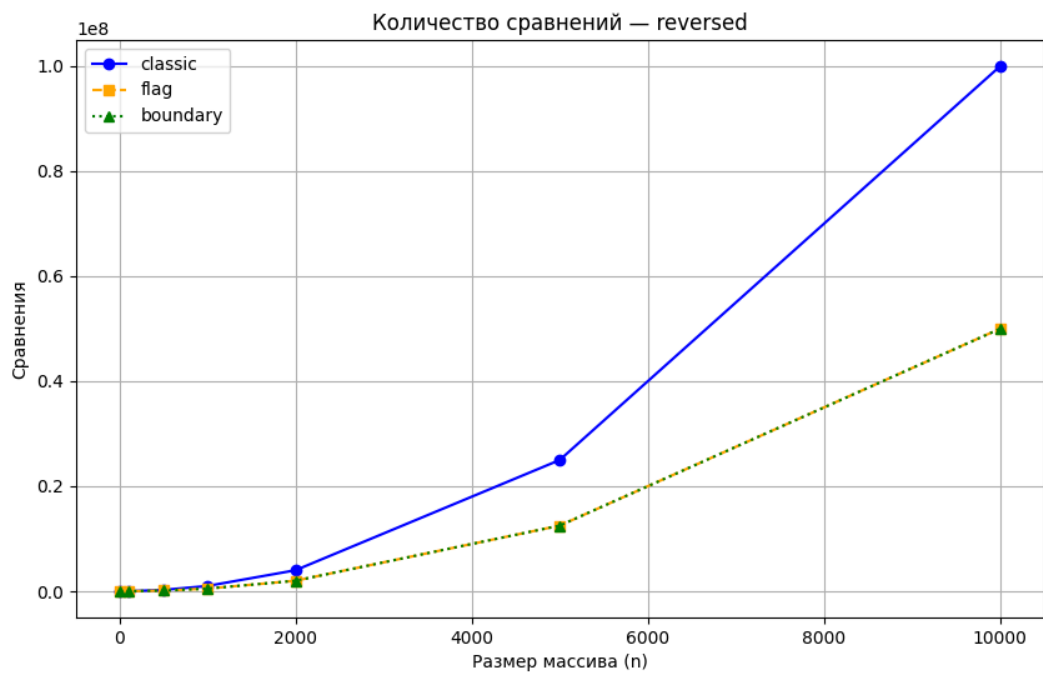
Случайные данные



Отсортированные данные

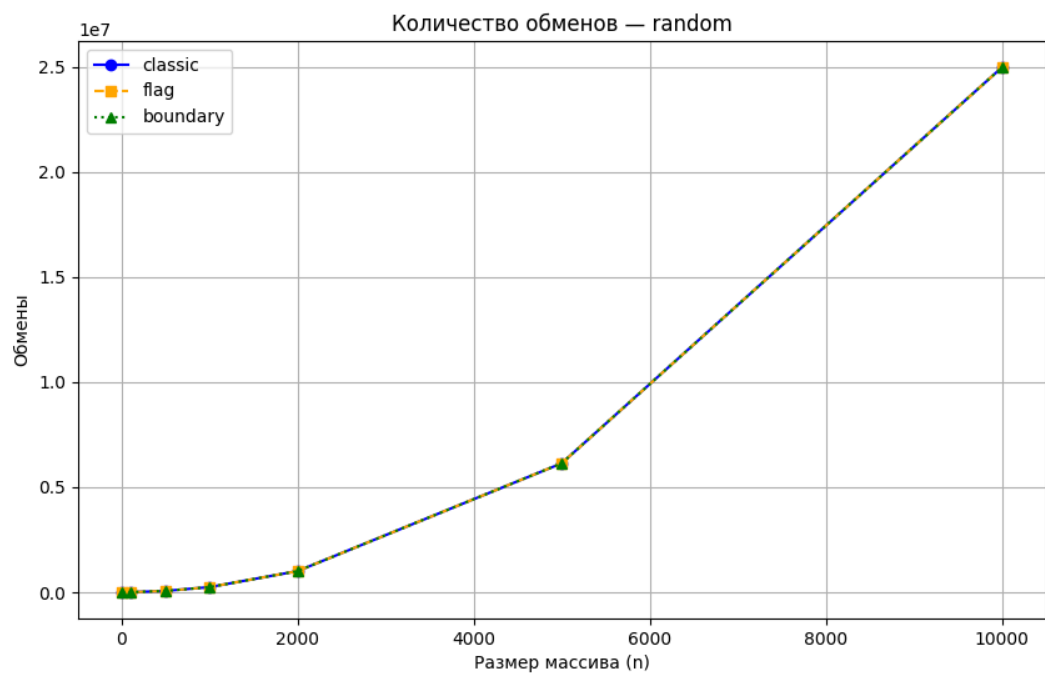


Обратный порядок

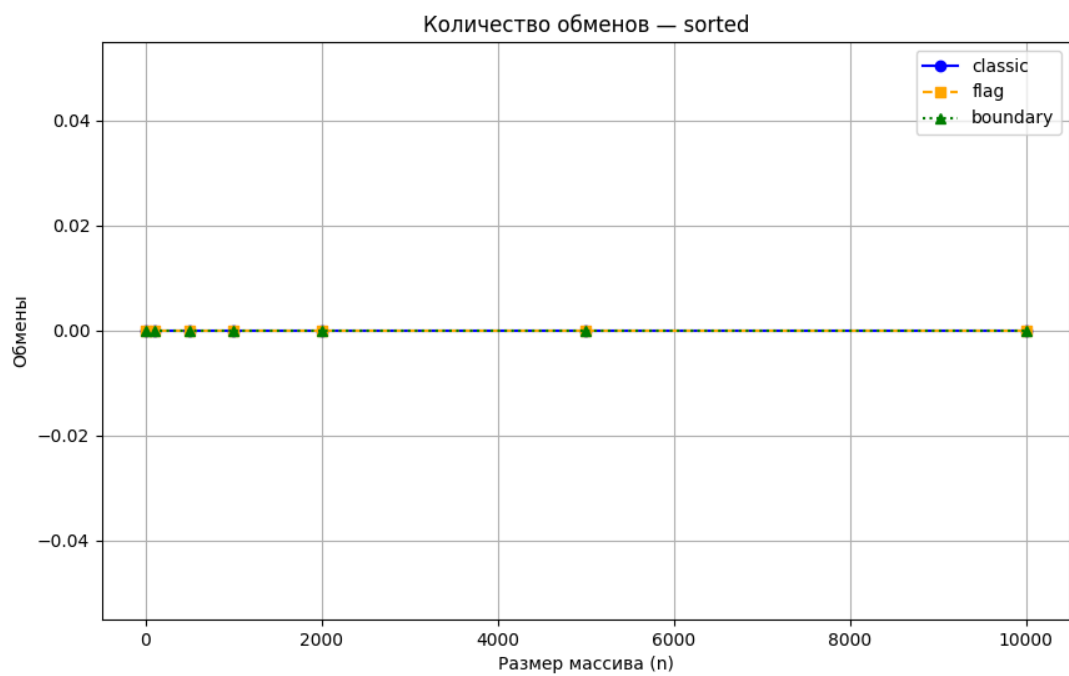


Графики количества обменов

Случайные данные

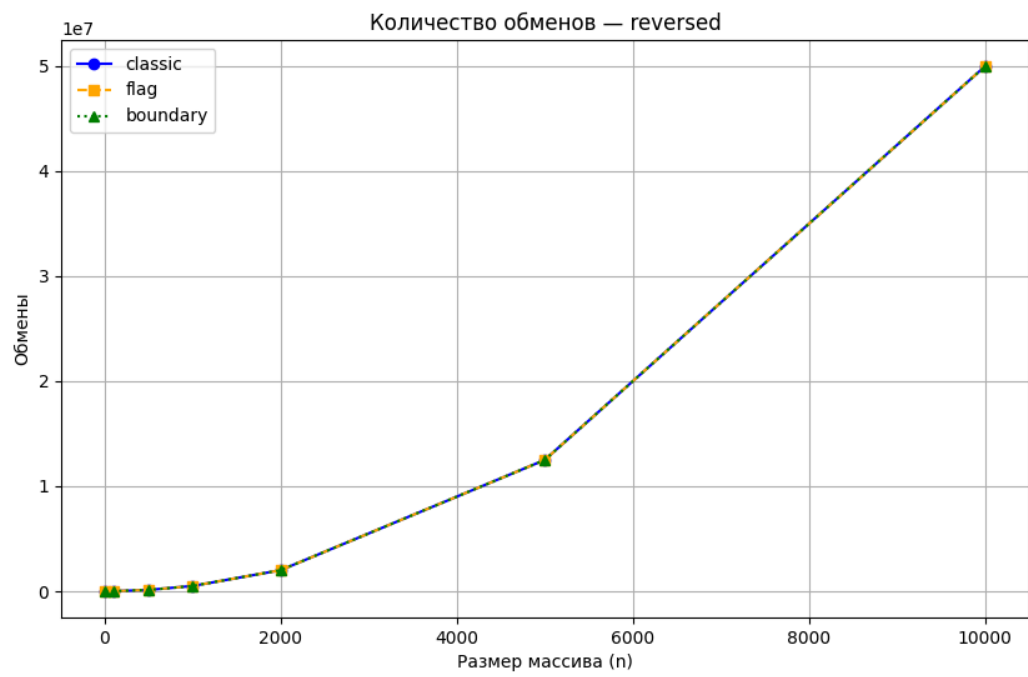


Отсортированные данные



typst compile report.typ

Обратный порядок



Анализ результатов

На основе полученных данных можно выделить несколько ключевых закономерностей.

Влияние типа входных данных. Наибольшее различие между алгоритмами проявляется на отсортированных данных. Классический пузырьёк выполняет $(n-1)^2$ сравнений независимо от порядка — при $n=10000$ это около 100 миллионов сравнений. Пузырёк с флагом и пузырьёк с границей на отсортированном массиве завершают работу после одного прохода, выполнив всего $n-1$ сравнений и 0 обменов. Это подтверждается данными: при $n=10000$ флаг и граница дают 9999 сравнений против 99 980 001 у классического.

Обратный порядок — худший случай для всех трёх алгоритмов. Количество обменов максимально и равно $n(n-1)/2$. При $n=10000$ это около 50 миллионов обменов у всех трёх алгоритмов — они совпадают, поскольку выполняют одни и те же перестановки, только с разным числом проходов.

Сравнение флага и границы на случайных данных. Оба алгоритма показывают схожие результаты — примерно вдвое меньше сравнений чем классический. Пузырёк с границей незначительно выигрывает по сравнениям за счёт сужения рабочей зоны.

Соответствие теоретической сложности $O(n^2)$. Графики времени и сравнений для классического пузырька имеют параболическую форму. При увеличении n в 2 раза (например с 5000 до 10000) время растёт примерно в 4 раза, что соответствует квадратичной зависимости. Для оптимизированных алгоритмов на случайных данных наблюдается аналогичная кривая, но с меньшим коэффициентом.

Выводы по работе

В ходе лабораторной работы были реализованы и исследованы три варианта пузырьковой сортировки: классический, с флагом досрочного завершения и с уменьшением рабочей границы.

Установлено, что классический пузырьрёк всегда выполняет фиксированное число сравнений вне зависимости от входных данных, что делает его неэффективным на частично упорядоченных массивах. Оптимизированные варианты значительно сокращают количество операций на отсортированных и почти отсортированных данных — в предельном случае до $O(n)$.

На случайных и обратно отсортированных данных все три алгоритма демонстрируют сложность $O(n^2)$, что подтверждается параболической формой графиков. Количество обменов одинаково для всех трёх алгоритмов при любом типе входных данных, поскольку сами перестановки от оптимизации не зависят.

Наиболее эффективным в общем случае является пузырьрёк с уменьшением рабочей границы — он сочетает досрочное завершение при отсутствии обменов и сокращение внутреннего цикла на каждой итерации.

Приложение. Исходный код

LR_1.cpp

```
#include <iostream>
#include <fstream>
//ofstream для вывода в файл
//ifstream для ввода в файл
#include <cstring> //memset
#include <chrono>
#include <cstdlib> // rand(), srand()
using namespace std;
//auto start = chrono::high_resolution_clock::now(); //auto -
автоматический тип данных
//подпространство имён(std пространство, chrono
вложенное пространство)
//high_resolution_clock - класс(пользовательский
тип данных)
//now - функция берущая время в данный
момент

enum DataType {
    RANDOM    = 0,
    SORTED    = 1,
    REVERSED  = 2
};

void generateArray(int arr[], int size, DataType type) {
    if (type == RANDOM) {
        for (int i = 0; i < size; i++) {
            arr[i] = rand() % 64001 - 32000; //rand() – функция без
параметров, возвращает псевдослучайное целое от 0 до RAND_MAX"
        } //берём от него остаток
64000 и получается число от 0 до 64000 и из него вычитается 32000:
Итоговый диапазон [-32000, 32000]
    }
    else if (type == SORTED) {
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }
    }
    else if (type == REVERSED) {
        for (int i = 0; i < size; i++) {
            arr[i] = size - 1 - i;
        }
    }
}
```

```
}
```

```
void classicBubble(int arr[], int size, long long &comps, long long
&swaps) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - 1; j++) {
            comps++; //на каждый j++ + одно сравнение
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swaps++;
            }
        }
    }
}
```

```
void flagBubble(int arr[], int size, long long &comps, long long
&swaps){
    for(int i = 0; i < size - 1; i++){
        bool swapped = false;
        for(int j = 0; j < size - 1 - i; j++){
            comps++;
            if(arr[j] > arr[j+1]){
                swap(arr[j], arr[j+1]);
                swaps++;
                swapped = true;
            }
        }
        if(!swapped) break;
    }
}
```

//самая крутая, сразу срезает всю часть массива, после верхней границы последнего обмена

//то есть, всё что дальше последнего swap(lastSwap) уже отсортировано и туда идти незачем

```
void boundaryBubble(int arr[], int size, long long &comps, long long
&swaps){
    int boundary = size - 1; // граница — до куда идём во внутреннем
    цикле
    while(boundary > 0){
        int lastSwap = 0; // индекс последнего обмена в этом
        проходе
        for(int j = 0; j < boundary; j++){
            comps++;
            if(arr[j] > arr[j+1]){
                swap(arr[j], arr[j+1]);
                swaps++;
            }
        }
        boundary = lastSwap;
    }
}
```

```

        lastSwap = j; // запоминаем, где был последний обмен
    }
}
boundary = lastSwap; // следующий проход – только до этого
места
    // если обменов не было, lastSwap = 0, boundary = 0, цикл
    завершится
}
}

// возвращает строку-имя для типа данных
const char* typeName(DataType type) {
    if (type == RANDOM)    return "random";
    if (type == SORTED)    return "sorted";
    if (type == REVERSED)  return "reversed";
    return "unknown";
}

int main() {
    auto seed =
chrono::high_resolution_clock::now().time_since_epoch().count(); //
Если не брать seed через замер времени, то srand возьмёт значение по
умолчанию(1) - srand(1) будет использоваться всегда и rand() будет
давать одну и ту же последовательность
    srand(seed); //отправная точка

    int sizes[] = {10, 100, 500, 1000, 2000, 5000, 10000};
    int numSizes = 7;

    int arr[10000];
    int buf[10000]; //buf нужен для копирования одного и того же
массива,
        //чтобы 3 разные сортировки его проработали не
меняя оригинал.

    ofstream out("results.csv");
    out << "Size;DataType;Algorithm;Comparisons;Swaps;Time_us\n";

    for (int s = 0; s < numSizes; s++) { //генирим массивы размерами
из size
        int n = sizes[s];

        for (int t = 0; t < 3; t++) {
            DataType type = (DataType)t; // приведение типа данных
(random, sorted, reversed)

```

```

generateArray(arr, n, type);

//1. Создаётся массив одного типа
//2. Все 3 сортировки его обрабатывают и записывают
данные в таблицу
//3. начинается новый цикл(создаётся массив следующего
типа, например reversed)

//алгоритм 1: классический пузырьк
long long comps = 0, swaps = 0;
memcpy(buf, arr, n * sizeof(int)); //sizeof оператор,
который возвращает размер типа данных
// в байтах, memcpy
Работает на уровне байт и ей нужно знать
// сколько байт
скопировать, если n = 10, то это 40 байт, поэтому
// n * 4(то есть
sizeof(int)) = 40
// можно писать n * 4,
но int не везде = 4(универсальность кода)

auto start = chrono::high_resolution_clock::now();
classicBubble(buf, n, comps, swaps);
auto stop = chrono::high_resolution_clock::now();
long long time_us =
chrono::duration_cast<chrono::microseconds>(stop - start).count();

out << n << ";" << typeName(type) << ";classic;"
    << comps << ";" << swaps << ";" << time_us << "\n";

//алгоритм 2: пузырьк с флагом
comps = 0; swaps = 0;
memcpy(buf, arr, n * sizeof(int));

start = chrono::high_resolution_clock::now();
flagBubble(buf, n, comps, swaps);
stop = chrono::high_resolution_clock::now();
time_us =
chrono::duration_cast<chrono::microseconds>(stop - start).count();

out << n << ";" << typeName(type) << ";flag;"
    << comps << ";" << swaps << ";" << time_us << "\n";

//алгоритм 3: пузырьк с границей
comps = 0; swaps = 0;
memcpy(buf, arr, n * sizeof(int));

```

```

        start = chrono::high_resolution_clock::now();
        boundaryBubble(buf, n, comps, swaps);
        stop = chrono::high_resolution_clock::now();
        time_us =
chrono::duration_cast<chrono::microseconds>(stop - start).count();

        out << n << ";" << typeName(type) << ";boundary;"
            << comps << ";" << swaps << ";" << time_us << "\n";
    }
}

out.close();
cout << "Done. Results are uploaded in results.csv" << endl;
return 0;
}

```